

ISMP 3D Anatomy — Backend

Resumen de modulos implementados

Stack: NestJS + Prisma + PostgreSQL + JWT

1. Arquitectura general

El backend esta construido con NestJS, un framework para Node.js que organiza el codigo en modulos. Cada modulo agrupa un controller, un service y sus DTOs. La base de datos es PostgreSQL y se accede mediante Prisma ORM.

Capa	Que hace	Ejemplo
Controller	Recibe el request HTTP y llama al service	AuthController, UsersController
Service	Contiene la logica de negocio	AuthService, UsersService
Guard	Intercepta el request y verifica permisos	JwtGuard, RolesGuard
DTO	Define y valida los datos del request	LoginDto, CreateUserDto
Prisma	Hace las queries a PostgreSQL	prisma.user.findMany()

Flujo de un request:

```
Request HTTP -> JwtGuard (token valido?) -> RolesGuard (rol correcto?) ->
Controller -> Service -> Prisma -> PostgreSQL -> Response JSON
```

2. PrismaModule

Es el modulo de conexion a la base de datos. Se configura una sola vez y queda disponible en toda la app gracias a @Global().

Archivos:

Archivo	Que hace
prisma/schema.prisma	Define los 7 modelos de la BD (User, Carrera, Materia, Cuestionario, Question, Attempt, Answer)
prisma/seed.ts	Carga datos iniciales: 2 carreras, 4 materias, 7 usuarios, 2 cuestionarios
src/prisma/prisma.service.ts	Extiende PrismaClient. Se conecta al iniciar y desconecta al cerrar
src/prisma/prisma.module.ts	Registra y exporta PrismaService como global

Decoradores clave:

`@Global()` -> hace que PrismaService este disponible en TODOS los modulos sin necesidad de importarlo cada vez

`@Injectable()` -> marca la clase como injectable por NestJS (patron Singleton)

Patron de diseno: Singleton — NestJS crea una sola instancia de PrismaService y la comparte en toda la app.

3. AuthModule

Maneja la autenticacion del sistema. Genera y verifica tokens JWT. Tiene dos endpoints publicos y expone los guards para que otros modulos los usen.

Endpoints:

Metodo	Ruta	Descripcion	Protegido
POST	/auth/login	Recibe email y password. Devuelve JWT + datos del usuario	No
GET	/auth/me	Devuelve el usuario autenticado a partir del token	JwtGuard

Archivos:

Archivo	Que hace
auth.controller.ts	Define las rutas POST /auth/login y GET /auth/me
auth.service.ts	Logica de login (verifica password con bcrypt, genera JWT) y me (busca usuario por id)
dto/login.dto.ts	Valida que el body tenga email valido y password de tipo string
guards/jwt.guard.ts	Lee el header Authorization, verifica el token JWT, adjunta el usuario al request
guards/roles.guard.ts	Lee el decorador @Roles(), verifica que el rol del usuario coincida
decorators/user.decorator.ts	Extrae el usuario del request de forma limpia con @CurrentUser()
decorators/roles.decorator.ts	Define @Roles() que etiqueta un endpoint con los roles permitidos

Decoradores clave:

Decorador	Donde se usa	Que hace
@Controller("auth")	AuthController	Define que esta clase maneja rutas que empiezan con /auth
@Post("login")	Metodo login	Responde a POST /auth/login
@Get("me")	Metodo me	Responde a GET /auth/me
@UseGuards(JwtGuard)	Metodo me	Ejecuta JwtGuard antes de que llegue al metodo
@Body()	Parametro dto	Lee el JSON del body del request
@CurrentUser()	Parametro user	Extrae el usuario del token ya verificado

@Injectable()	Guards y Services	Marca la clase como inyectable por NestJS
---------------	-------------------	---

Patrones de diseno: Proxy (JwtGuard intercepta requests), Chain of Responsibility (guards en cadena), Singleton (services).

4. UsersModule

CRUD completo de usuarios. Solo accesible por el rol ADMIN. Aplica JwtGuard y RolesGuard en toda la clase del controller.

Endpoints:

Metodo	Ruta	Descripcion	Rol requerido
GET	/users	Lista todos los usuarios. Filtros: ?role= y ?carreraId=	ADMIN
GET	/users/:id	Ver datos de un usuario especifico	ADMIN
POST	/users	Crear usuario nuevo. Hashea la password automaticamente	ADMIN
PATCH	/users/:id	Editar campos del usuario. Si viene password la hashea	ADMIN
DELETE	/users/:id	Borrar usuario	ADMIN

Archivos:

Archivo	Que hace
users.controller.ts	Define los 5 endpoints. Aplica guards y roles en toda la clase
users.service.ts	Logica de cada operacion. Verifica email duplicado, hashea passwords, nunca devuelve password
dto/create-user.dto.ts	Valida: name (string), email (email valido), password (min 6 chars), role (enum), carreraId (opcional)
dto/update-user.dto.ts	Igual que create pero todos los campos son opcionales (?)
users.module.ts	Registra el modulo e importa AuthModule para usar los guards

Decoradores clave:

hace		
Importa TODOS los endpoints del controller		
Indica que solo ADMIN puede acceder a cualquier endpoint		
@Patch("/:id")	@Delete("/:id")	Metodos del controller
Indica el :id de la URL (ej: /users/abc123)		
Indica los query params de la URL (ej: ?role=DOCENTE)		
Indica que se rellenen los campos del body automaticamente		

5. Niveles de seguridad implementados

5.1 Autenticacion con JWT

Cuando el usuario hace login el backend genera un JWT firmado con una clave secreta (JWT_SECRET). El token contiene: id del usuario, email y rol. Dura 12 horas.

```
Token = datos_del_usuario + firma_con_JWT_SECRET -> si alguien lo modifica la firma no coincide y es rechazado
```

5.2 Autorizacion por roles

Cada endpoint tiene un nivel de acceso definido por @Roles(). El RolesGuard verifica que el rol del token coincida con el requerido.

Situacion	Respuesta	Codigo HTTP
Request sin token	Token no encontrado	401 Unauthorized
Token invalido o expirado	Token invalido o expirado	401 Unauthorized
Token valido pero rol incorrecto	No tienes permiso para realizar esta accion	403 Forbidden
Email o password incorrecto en login	Credenciales incorrectas	401 Unauthorized
Email ya en uso al crear usuario	El email ya esta en uso	409 Conflict
Usuario no encontrado por ID	Usuario con id X no encontrado	404 Not Found
Datos invalidos en el body	Lista de errores de validacion	400 Bad Request

5.3 Proteccion de datos sensibles

- + Passwords hasheadas con bcrypt (factor 10) — nunca se guardan en texto plano
- + passwordHash nunca se devuelve en ninguna respuesta (uso de select en Prisma)
- + JWT_SECRET en variable de entorno .env — nunca en el codigo
- + ValidationPipe con whitelist:true — campos extra del body son ignorados automaticamente
- + Emails unicos — no se puede registrar dos usuarios con el mismo email

6. Todas las rutas disponibles

Metodo	Ruta	Descripcion	Acceso
POST	/auth/login	Login con email y password	Publico
GET	/auth/me	Usuario autenticado actual	Token JWT

GET	/users	Listar usuarios (filtros: ?role, ?carreraId)	ADMIN
GET	/users/:id	Ver un usuario	ADMIN
POST	/users	Crear usuario	ADMIN
PATCH	/users/:id	Editar usuario	ADMIN
DELETE	/users/:id	Borrar usuario	ADMIN

Proximos modulos a implementar: Cuestionarios (DOCENTE/ADMIN) e Intentos (ESTUDIANTE).